

Object Oriented Programming with Java

25.3 - Faculty of Computing



Take Home Activity 07

Answer all questions.

1. Create a class called **Calculator** with the following methods:
 - i. add(int a, int b) – returns the sum of two integers and prints: "Int sum: [result]"
 - ii. add(double a, double b) – returns the sum of two doubles and prints: "Double sum: [result]"
 - iii. add(int a, int b, int c) – returns the sum of three integers and prints: "Three-int sum: [result]"
 - iv. add(String a, String b) – concatenates two strings and prints: "String join: [result]"
 - a. In **main()**:
Create a Calculator object and call all the methods
2. Create a class called **Printer** with the following methods:
 - i. printData(String text) – prints: "Printing text: [text]"
 - ii. printData(int number) – prints: "Printing number: [number]"
 - iii. printData(String text, int copies) – prints: "Printing [copies] copies of: [text]"
 - iv. printData(boolean isDuplex) – prints: "Duplex printing: [isDuplex]"
 - a. In **main()**:
Create a Printer object and call all the methods
3. Create a **class** called **Animal** with the following fields and methods:
name of type String
makeSound() – prints: "[name] makes a generic sound"
 - a. Create three subclasses that extend Animal and override makeSound():
 - Dog – prints: "[name] says: Woof!"
 - Cat – prints: "[name] says: Meow!"
 - Bird – prints: "[name] says: Tweet!"
 - b. In main() Part 1 (direct calls):
 - Create a Dog, set name to "Rex", call makeSound()
 - Create a Cat, set name to "Luna", call makeSound()
 - Create a Bird, set name to "Tweety", call makeSound()
 - c. In main() Part 2 (Dynamic Dispatch):
 - Upcast: Animal a1 = new Dog(); set name to "Bruno", call a1.makeSound()

- Upcast: Animal a2 = new Cat(); set name to "Kitty", call a2.makeSound()
- Upcast: Animal a3 = new Bird(); set name to "Sky", call a3.makeSound()

4. Create a class called **Shape** with the following methods:

draw() – prints: "Drawing a generic shape"

area() – prints: "Area: 0.0"

a. Create three subclasses that extend Shape and override both methods:

Circle – field radius (double). draw() prints: "Drawing a Circle". area() = $\pi \times r^2$, prints: "Area: [result]"

Rectangle – fields width, height (double). draw() prints: "Drawing a Rectangle".

area() = $w \times h$, prints: "Area: [result]"

Triangle – fields base, height (double). draw() prints: "Drawing a Triangle". area() = $0.5 \times b \times h$, prints: "Area: [result]"

b. In main() - Dynamic Dispatch with an array:

Create a Shape[] array of size 3 ← this IS upcasting (each slot holds a Shape reference)

shapes[0] = new Circle() with radius 5

shapes[1] = new Rectangle() with width 4, height 6

shapes[2] = new Triangle() with base 3, height 8

Loop through and call draw() and area() on each element

5. Create a class called BackAccount with the following fields:

accountNumber of type String

holderName of type String

balance of type double

a. Add the following methods - use Method Overloading for deposit():

showDetails() – prints: "Account: [accountNumber] | Holder: [holderName] | Balance: [balance]"

deposit(double amount) – adds amount, prints: "Deposited [amount]. New balance: [balance]"

deposit(double amount, String currency) – if currency is "USD" multiply by 300, prints: "Deposited [amount] [currency] = LKR [converted]. New balance: [balance]"

b. Create two subclasses that extend BankAccount and override showDetails():

SavingsAccount – adds interestRate (double). Override showDetails() to append "| Type: Savings | Rate: [interestRate]%" . Add applyInterest() – balance += balance × rate/100, prints: "Interest applied. New balance: [balance]"

LoanAccount – adds loanAmount (double). Override showDetails() to append "| Type: Loan | Loan: [loanAmount]". Add repayLoan(double amount) – reduces loanAmount, prints: "Loan repaid: [amount]. Remaining: [loanAmount]"

- c. In main() Part 1 (direct calls):
 - Create a SavingsAccount, set accountNumber "SA-001", holderName "Alice", balance 5000, interestRate 3.5
 - Call showDetails(), deposit(1000.0), deposit(50.0, "USD"), applyInterest()
 - d. In main() Part 2 (Dynamic Dispatch):
 - Upcast: BankAccount b1 = new SavingsAccount() with the same values above
 - Call b1.showDetails()

 - Upcast: BankAccount b2 = new LoanAccount() set accountNumber "LA-002", holderName "Bob", loanAmount 20000
 - Call b2.showDetails()
6. Create a class called SecurityAlert with the following fields:
- threatId of type String
 - source of type String
 - severity of type String (e.g., "Low", "Medium", "High", "Critical")
- a. Add the following methods - use Method Overloading for logAlert():
 - logAlert() – prints: "[ALERT] ID: [alertId] | Source: [source] | Severity: [severity]"
 - logAlert(String extraInfo) – prints the above and appends "| Info: [extraInfo]"
 - logAlert(String extraInfo, boolean sendEmail) – prints the above; if sendEmail is true also prints: "Email notification sent for [alertId]"
 - raiseAlert() – prints: "!! SECURITY ALERT !! [alertId] from [source]"
 - b. Create three subclasses that extend SecurityAlert and override raiseAlert():
 - MalwareAlert – adds malwareType (String). Override raiseAlert() prints: "[MALWARE] [malwareType] detected from [source]!". Add quarantine() – prints: "Quarantining [malwareType] from [source]...."
 - PhishingAlert – adds targetEmail (String). Override raiseAlert() prints: "[PHISHING] Attack targeting [targetEmail] from [source]!". Add blockEmail() – prints: "Blocking phishing email targeting: [targetEmail]"
 - DDoSAlert – adds packetCount (int). Override raiseAlert() prints: "[DDoS] [source] - [packetCount] packets!". Add mitigate() – prints: "Mitigating DDoS from [source] -[packetCount] packets dropped"
 - c. In main() -Part 1 (Overloading demo):
 - Create a MalwareAlert, set alertId "ALT-001", source "192.168.1.5", severity "Critical", malwareType "Ransomware"
 - Call logAlert() - no extra info
 - Call logAlert("Encrypted 3 files") - with extra info

Call logAlert("Encrypted 3 files", true) - with email notification
Call raiseAlert() and quarantine()

- d. In main() - Part 2 (Dynamic Dispatch):
Create a SecurityAlert[] array of size 3 ← upcasting
alerts[0] = new MalwareAlert() set alertId "ALT-001", source "192.168.1.5",
severity "Critical", malwareType "Ransomware"
alerts[1] = new PhishingAlert() set alertId "ALT-002", source "evil.com",
severity "High", targetEmail "admin@nsbm.ac.lk"
alerts[2] = new DDoSAlert() set alertId "ALT-003", source "10.0.0.99", severity
"Critical", packetCount 50000
Loop through the array and call logAlert() and raiseAlert() on each element